

LANGAGE C# ET ÉCOSYSTÈME .NET CORE

I. LE LANGAGE C# : NATURE ET ÉVOLUTION

Le C# (prononcé "See Sharp") est défini comme un langage simple, moderne et orienté objet. Sa proximité syntaxique avec la famille des langages C le rend immédiatement accessible aux développeurs Java ou JavaScript.

HISTORIQUE DES VERSIONS

Le langage a suivi un cycle de mise à jour régulier depuis son lancement : | Version | Date de sortie || ----- | ----- || 1.0 | Janvier 2002 || 2.0 | Novembre 2005 || 3.0 | Novembre 2007 || 4.0 | Avril 2010 || 5.0 | Août 2012 || 6.0 | Juillet 2015 || 7.0 | Mai 2018 || 8.0 | Septembre 2019 |

CARACTÉRISTIQUES FONDAMENTALES

- **Fortement typé** : Chaque variable doit se voir attribuer un type spécifique (entier, booléen, chaîne, etc.) lors de sa déclaration.
- **Sensibilité à la casse** : Le langage distingue les majuscules des minuscules dans les déclarations.
- **Gestion des variables** : Distinction entre les variables membres (champs) et les variables locales, incluant le support des constantes et des types "Nullable".

2. L'ÉCOSYSTÈME .NET CORE ET .NET 5

.NET CORE : LA PLATEFORME MULTIPLATEFORME

.NET Core est une plateforme généraliste open source permettant de générer des applications pour le Cloud, l'Internet des Objets (IoT) et divers appareils.

- **Compatibilité OS** : Windows (7 SP1+, 10, Server 2012 R2+), macOS (10.13+) et de nombreuses distributions Linux (RHEL, CentOS, Debian, Ubuntu, etc.).
- **.NET Standard** : Il s'agit d'une spécification officielle des API .NET destinée à assurer l'uniformité entre toutes les implémentations .NET.
- **Types d'applications (v3.0)** : Console, sites web ASP.NET MVC, API, ainsi que les applications desktop Windows (UWP, WPF, Windows Forms).

VERS L'UNIFICATION AVEC .NET 5

Annoncé pour novembre 2020, .NET 5 a pour objectif d'unifier l'univers .NET.

- **Universalité** : Accessible pour Windows, Linux, macOS, iOS, Android, WebAssembly et plus.
- **Convergence** : Exploitation du meilleur des technologies .NET Core, .NET Framework, Xamarin et Mono.
- **Rythme de sortie** : Des versions majeures chaque année, les versions paires bénéficiant d'un support à long terme (LTS).

3. OUTILS DE DÉVELOPPEMENT ET INSTALLATION

Le développement en C# repose sur une hiérarchie d'outils, du plus basique (CLI) au plus complexe (IDE).

L'INTERFACE DE LIGNE DE COMMANDE (CLI) DOTNET

C'est le fondement sur lequel reposent les autres outils. Les commandes essentielles sont :

- `dotnet --version` : Déterminer la version installée.
- `dotnet --info` : Obtenir des détails sur le système, le SDK et le runtime.
- `dotnet new --list` : Lister les modèles de projets disponibles (ex: console, webapi, mvc).
- `dotnet new console -o NomProjet` : Créer un nouveau projet console dans un répertoire spécifique.
- `dotnet build` : Compiler le projet.
- `dotnet run` : Exécuter l'application.

ENVIRONNEMENTS DE DÉVELOPPEMENT (IDE ET ÉDITEURS)

- **Visual Studio Code** : Éditeur de code source léger et multiplateforme. Il nécessite l'extension "C# for Visual Studio Code" pour le développement .NET Core.
- **Visual Studio (IDE)** : Environnement complet disponible en plusieurs versions :
- **Community** : Gratuite.
- **Professionnelle et Entreprise** : Payantes, pour des besoins avancés.
- **Online** : Environnement basé sur le cloud.

4. ANATOMIE D'UN PROJET C#

Un projet C# standard, tel qu'ouvert dans Visual Studio Code, se compose de deux éléments pivots :

- **Le fichier .csproj** : Fichier de configuration au format XML détaillant le framework cible (ex: netcoreapp3.1) et le type de sortie (ex: Exe).
- **Le fichier Program.cs** : Contient le code source. Tout programme C# doit inclure une méthode Main. C'est le point d'entrée : le programme commence à l'appel de cette méthode et s'arrête à la fin de son bloc d'instructions.

INTERACTION AVEC LA CONSOLE

Au début de l'apprentissage, l'interface utilisateur est principalement la console :

- **Écriture** : `Console.Write()` (reste sur la ligne) et `Console.WriteLine()` (retour à la ligne).
- **Lecture** : `Console.ReadLine()` retourne une chaîne de caractères (string) fournie par l'utilisateur.
- **Formatage** : Possibilité d'utiliser des index {0}, {1}... pour insérer plusieurs variables dans une même ligne.

5. DÉBOGAGE ET MAÎTRISE DU CODE

Le débogage est une phase critique de la programmation. Les outils modernes offrent des fonctionnalités avancées pour inspecter le comportement du code.

MÉCANISMES DE DÉBOGAGE

- **Points d'arrêt (Breakpoints)** : Placés dans la bande grise verticale, ils stoppent l'exécution pour permettre l'inspection.
- **Flèche jaune** : Indique la prochaine action à exécuter. Elle peut être déplacée manuellement pour sauter des instructions ou revenir en arrière.
- **Espions (Watch)** : Permettent de surveiller des expressions ou variables en temps réel. Une valeur qui change s'affiche en rouge.

RACCOURCIS CLAVIER ESSENTIELS (VISUAL STUDIO)

Raccourci, Action

F5, Lancer l'exécution avec débogage.

CTRL + F5, Lancer l'exécution sans débogage.

F11, Continuer l'exécution pas à pas.

F7 / Maj + F7, Basculer entre l'affichage Code et le mode Design (graphique).

"CTRL + ,", Recherche intelligente dans la solution (définitions, types, méthodes).

CTRL + K + T, Afficher la hiérarchie d'appel d'un élément.

GESTION AVANCÉE DU CODE

- **IntelliSense** : Complétion automatique suggérant les objets et méthodes à mesure que le développeur tape les premières lettres.
- **Surlignage des références** : La sélection d'un élément met en évidence toutes ses occurrences dans le fichier.
- **Multi-écrans** : Depuis la version 2010, Visual Studio permet de détacher et de déplacer toutes les fenêtres (Explorateur de solutions, Propriétés, Toolbox) en dehors de l'environnement principal, même sur d'autres écrans.